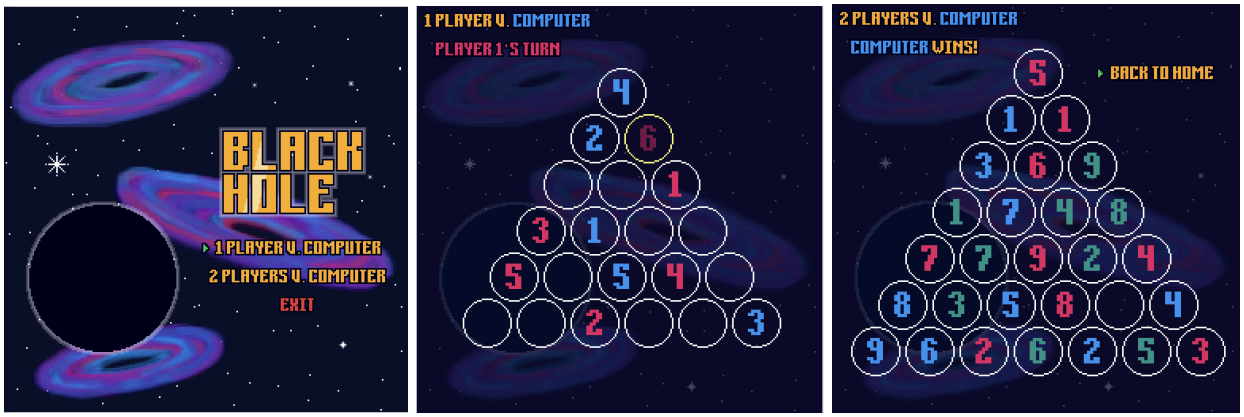
Asha Mavankal
7 Apr 2026
CS 2340.HN1

CS 2340 Term Project Report: Black Hole



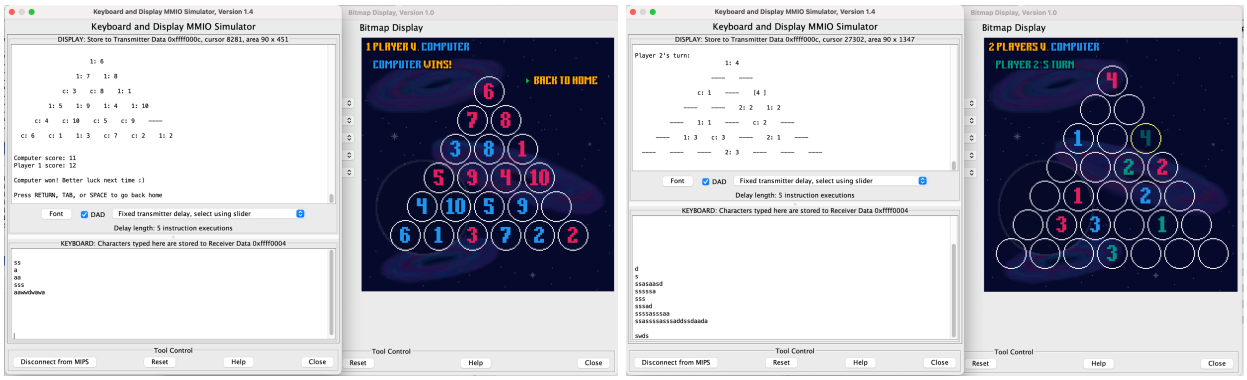


Table of Contents

Program Description....2
Features....2
Program Structure....3
Challenges....3
What I learned....4
Algorithms & techniques used....4
Home Screen Animation....4
Efficient Graphics Rendering....5
Input Handling....5
Board Representation & Traversal....5
Printing the Board....6
Computer Turn Algorithm....6
Game Loop Algorithm....6

Program Description

← *Back to Table of Contents*

This program uses MARS' MMIO Keyboard & Display Simulator and Bitmap Display to run a game of black hole against a computer opponent using both GUI- and text-based game rendering. It supports both a single-player mode (1 player v. computer) and a two-player mode (2 players v. computer) .

Once the Bitmap Display and MMIO Keyboard & Display Simulator are opened, the core program flow starts at the animated home screen, run by the `home` subroutine, in which a user can use the WASD keys (subbing in as arrow keys) in the MMIO keyboard input bank to select a game mode or exit the program—the options are displayed in both the bitmap display (GUI) and MMIO text display (console).

Once a user has selected a game mode, the program moves on to the `game` subroutine, which renders a game in both the GUI and console. It allows players to use the WASD keys to navigate the game board, responsively drawing the user's cursor on the GUI and displaying a text-based rendering of the board to the console, which allows for intuitive spot selection.

Once all players have selected a spot for their values, the `game` subroutine has the computer take its turn by selecting a random spot to place its next number in, and this loop continues until there is 1 empty spot left on the board, at which point the game is complete.

Once the game is complete, the game subroutine calculates the scores for each player and the computer, and displays the scores to the console. It also displays a message denoting who won to both the GUI and console, and then it allows the user to return back to the home screen and repeat the game loop by hitting enter.

Features

- 2 game modes: 1-player v. computer, 2-player v. computer
- Animated home screen with interactive menu options
- Arrow-key-based navigation with WASD & selection with RETURN, SPACE, or TAB
- GUI rendering of the game board with the Bitmap Display
- TUI rendering of game board in MMIO Display
- Interactive and responsive board spot selection using a visual cursor in both GUI and text console
- Visual indication in both GUI and Console of whose turn it is
- Score calculated & results displayed to both GUI & text console
- Board reset & program loop allowing repeated play until user chooses to exit
- Instructions for how to set up MMIO & Bitmap printed to standard output at start of program

Program Structure

The program entry point is the `main` subroutine in `driver.asm`, which controls the main program loop. The main program logic is organized around 2 subroutines, `home` and `game` (in `home.asm` and `game.asm`), which implement the home screen and gameplay, respectively.

Supporting subroutines are organized into helper files and included through `modules-wrapper.asm`, while GUI data is included in `graphics-wrapper.asm`. These 2 files are called "wrappers" because they use the `.include` directive to include the contents of source files organized in the `modules/` and `graphics/` subdirectories in a wrapper in the project root so that MARS assembles those files.

The `modules/` directory organizes the program's helper subroutines into 4 logical modules.

- `display/` contains subroutines used to display data to the MMIO Display.
- `game/` contains subroutines used to do calculations and complete tasks in the game subroutine.
- `gui/` contains subroutines used to display data to the Bitmap display.
- `input/` contains subroutines to validate user input.

Challenges

← Back to Table of Contents

The two biggest challenges I faced in writing this program were maintaining proper subroutine scope and register discipline and implementing board traversal via the up/down keys (`W`/`S`).

My program is quite modular, especially the game loop, and it involved a lot of subroutines calling subroutines, with the callee subroutines needing data from the callers. When I first started coding the program, I was a bit lazy and didn't want to worry about saving passed arguments to the stack or saved variables, so I made subroutines that would directly modify and read data from their calling subroutines (like reading `$s0` or modifying `$s3`).

However, as the program grew I had to change this approach because it made the program *incredibly* difficult to debug. Since I hadn't used strict register discipline (didn't ensure saved registers are saved across function calls, assumed arguments would be maintained once a function returns, relied on temporary registers being the same after function calls, etc..), once I started tweaking subroutines or adding functionality, that flimsy balance crashed and burned, and I had a horrible time debugging. I had to go back and refactor many of my subroutines and calls to subroutines to properly use arguments to pass data, and ensure that any data I wanted saved across subroutine calls were stored in saved registers. The only subroutine I left without clean arguments and returns was `get_user_selection` in `game.asm`, because it needed access to a lot of saved registers in the `game` subroutine and was a helper subroutine unique to `game`, so I didn't need to worry about it being too confused by calls from multiple places.

Another big challenge I faced in writing this program was in coding the logic to traverse the game board using the up/down keys. My game board is stored in memory as a 1 dimensional

array of bytes, so in order to to go up a row or down a row from a certain index, I needed to know its row and column in order to calculate the above/below circles, and I needed to decide which of the two upper or lower spots to prioritize when traversing.

In order to get the row number, my first thought was a mathematical approach (*row = (int) (((sqrt(8 * i + 1)) - 1) / 2)*), but floating point operations are expensive, and there are only max 7 rows (for a 2-player game), so I decided to just hard code a subroutine to return the row number given the index to solve that issue. Ideally, I would've had an algorithm like I had for the left/right keys that keeps checking up/down in a loop until an open spot is found, but I eventually compromised on that idea because it would be complicated to implement with the time I had (I would probably need a recursive function), so I just had the up/down keys check their immediate neighbors for free spots.

Some other challenges I faced:

- Balancing animation loop sleeps with responsive input (found a compromise, explained in algorithms & techniques)
- Organizing the GUI data in a space efficient way, especially the animation frames (also found a solution, listed in algorithms & techniques)

What I learned

← Back to Table of Contents

One of the most important things I learned from writing a project as big as this is how very important it is to maintain register discipline and clean code organization & modularization. As I already mentioned earlier, I suffered much as my program started to grow from not being careful about how I used registers between and during subroutine calls, but once I cleaned up my code and maintained register discipline, debugging became much easier and faster, especially because my code was chunked into different modules.

Another thing I learned is how I can effectively make use of the stack. Usually I just thought of the stack as a way to store the frame pointer and return address in non-leaf functions, but in this program I learned how much more effectively the stack can be used. For example, it can be used to save arguments or variables across a subroutine call without needing to use a saved register.

The most interesting way I learned to use the stack in my program was in my display_number subroutine, which prints a number to the MMIO Display. Its basic algorithm required a stack, since I'd calculate a number's digits from the least significant bit, but I wanted to print them starting from the most significant bit. So, I used the stack to store a local array that behaves as a stack by decrementing the stack pointer to push data, and incrementing it to pop data. By writing that subroutine, I realized how powerful the stack can be as a tool to store things like local arrays and such.

Algorithms & techniques used

← *Back to Table of Contents*

Home Screen Animation

The home screen animation was implemented by looping through an array of frame differences and updating the screen with only the different pixels after a certain time interval. In contrast to redrawing the entire frame each cycle, this is much more time and memory efficient. It also is able to read user input to select from the menu options by polling the MMIO keyboard control register in between frames, and responding to any input that is detected by changing the user's selected option or exiting the subroutine and returning their selection.

One challenge when designing the home screen animation was the fact that the time between frame changes is 150 ms, which can feel a bit slow when reading and responding to user input. I originally only polled the keyboard register every frame change, so every 150 ms, but I also wanted a slightly more responsive input reading system, so I found a compromise by setting the sleep interval between input polls to be 10 ms, and keeping a poll counter and only moving to the next animation frame after the count reached 15. That way, if a user changes their selection, the arrows would be updated almost instantaneously, but the frame lag between actual animation frames would stay the same.

Efficient Graphics Rendering

Similar to how the program uses frame differences in the home animation to render the graphics more efficiently, in the game loop it also uses frame differences that are specific to certain assets, like player turn messages or numbers on the board. By utilizing these pixel differences, the program only updates exactly the pixels it needs to, thus avoiding wasting time by redrawing the whole frame each time.

In order to render the assets that were aligned with spots on the board (like to change the color of a circle on the board or draw a number), an array of offsets was stored of the offsets for each index in the game board, and an array of offsets for an aligned asset that was originally aligned at 0 is added to those offsets to align it at the correct position.

Input Handling

The program only accepts a few types of valid inputs: the WASD keys (lowercase or uppercase) and the RETURN, TAB, or SPACE keys to enter. Input validation is done via helper subroutines that check the input against a list of accepted values (e.g. `is_go_character` checks if input is a RETURN, TAB, or SPACE) and return 0 if there is no match, and 1 if there is a match. The program then responds to the input based on the outcome of those subroutine calls, enabling it to handle invalid input.

Since input is read from the MMIO keyboard, it is read exclusively through polling. In the home animation, input was read in a non-blocking manner (i.e. the program did not stop to wait for input if it was not found when polled), but in the game loop input was read in a blocking manner

(i.e. the program waited until there is input) since it is what determines the program's next steps.

Board Representation & Traversal

The board is represented as a 1-dimensional array of 56 bytes; this is so that it could hold either 21 or 28 indices, depending on the game mode. Each conceptual index in the board is 2 bytes long: the first byte holds the player whose number the spot holds (`-1` = empty, `0` = computer, `1` = player 1, `2` = player 2), and the second byte holds the value of the spot (an integer `[1, 10]` for 1-player mode, or `[1, 9]` for 2-player mode). Indices in the board thus must be accessed by multiplying the index by 2 to access the spot owner, and adding 1 to that to get the spot's value.

The board was traversed left-to-right with simple loops that kept moving left or right in the game board, wrapping around, until a free spot was found or it looped back to itself. Up and down traversal was more complicated, and was done by calculating the upper right, upper left, bottom right, and bottom left indices relative to the cursor by adding or subtracting the 1-indexed row number of the cursor and adding or subtracting 1. The 1-indexed row number was found via a loop with hard-coded row-index values.

Printing the Board

The board was printed by a simple algorithm that printed *(# of rows - row #) * 4* spaces, then *n* blocks of 8 character-wide chunks representing a space on the board, where *n* is the row #. The 8-character wide chunks had 4 characters of actual data and 4 characters of space padding. The 4 characters that represented the actual spot on the board were then determined by the value of the spot; if it's empty, its value is `----`, if it's full it will list its owner and value (e.g. `c: 5`), and if the spot is a temporary spot (i.e. it is where the user's cursor is on the board), its potential value is listed in brackets (e.g. `[5 ]`).

Computer Turn Algorithm

The computer's next free spot was determined by generating a random integer *i* in the range of *[0, number of spots remaining),* and choosing the *i*th empty spot on the board when traversing it from 0. The number of spots remaining on the board is tracked throughout the game loop.

Game Loop Algorithm

The game uses a structured loop that first has the players take their turns, then the computer takes its turn, and it loops until there is 1 empty spot left on the board, at which point it breaks from the loop and calculates the final scores and game results.